

Weekly Progress Report — Week 2

Cloud-Based Research Paper Summarization Platform

Shreyas Sankpal · Pranjal Mishra · Yang Zheng · Kerry Huang

April 13, 2026

1. Executive Summary

Week 2 concluded the design phase of the project. Deliverables include a layered system architecture aligned with the AWS Well-Architected Framework, a finalized serverless AWS topology, a documented access-pattern specification for the primary data store, a preliminary risk register, and an initial set of service-level objectives. Implementation commences in Week 3.

2. Project Overview

The platform enables academic users to search open-access research repositories such as arXiv, Semantic Scholar, and Crossref; retrieve full-text PDFs; and generate structured summaries covering objectives, methodology, results, limitations, and contributions. The system is designed as an event-driven, serverless-first application deployed on Amazon Web Services, with design objectives of low idle cost, asynchronous long-running workloads, and operational simplicity.

3. Week 1 Recap

During Week 1 the team surveyed reference architectures for document-ingestion and large-language-model summarization pipelines, benchmarked three candidate LLM providers (OpenAI GPT-4o, Google Gemini 1.5, and Anthropic Claude via Amazon Bedrock) on sample papers, and selected Amazon Bedrock as the LLM surface. The selection was motivated by native AWS Identity and Access Management (IAM) authorization, the absence of an external API key requiring manual rotation, and built-in CloudWatch integration.

4. Week 2 Deliverables

The following artifacts were produced during the reporting period:

1. Version 1.0 of the high-level architecture document.
2. The AWS topology diagram (Figure 1).

3. A single-table access-pattern specification for Amazon DynamoDB.
4. The risk register presented in Section 8.
5. Preliminary service-level objectives presented in Section 9.
6. Selection of AWS Cloud Development Kit (CDK) in TypeScript as the Infrastructure-as-Code tool for the project.

5. Cloud Architecture

The system is organized into four functional layers, implemented entirely with AWS managed services.

Edge and frontend. Amazon Route 53 provides DNS resolution. Amazon CloudFront serves the static Next.js frontend from a private Amazon S3 bucket protected by a CloudFront Origin Access Identity, providing TLS, custom-domain support, and edge caching.

Authentication. An Amazon Cognito user pool manages signup and login. Amazon API Gateway enforces authentication through a Cognito JWT Authorizer; requests bearing an invalid or missing token are rejected at the gateway before any Lambda function executes.

Synchronous API. Amazon API Gateway routes authenticated requests to three AWS Lambda handlers: `searchFn` queries external academic APIs; `submitJobFn` validates the request, records a job in DynamoDB, enqueues an SQS message, and returns `202 Accepted` with a generated job identifier; `getSummaryFn` returns the status and content of a summary.

Asynchronous summarization pipeline. Amazon Simple Queue Service (SQS), configured with a dead-letter queue and a redrive policy, triggers an AWS Step Functions state machine comprising five steps: `FetchPDF` downloads the paper from its source and stores it in S3; `ExtractText` converts the PDF to raw text; `Chunk` produces overlapping logical text chunks; `MapSummarize` invokes Amazon Bedrock (Claude Sonnet) on each chunk in parallel; and `ReduceSummary` consolidates the chunk summaries into the final structured output written to DynamoDB. The use of Step Functions avoids the 15-minute single-Lambda execution limit, provides native retries per step, and yields a visual execution history useful for debugging.

Data. Amazon DynamoDB serves as the primary data store with a single-table design. The partition key is `USER#{userId}`; sort-key prefixes denote the entity type: `PROFILE#`, `JOB#{jobId}`, and `SUMMARY#{timestamp}`. A global secondary index keyed on `PAPER#{doi}` enables cross-user deduplication of summaries for the same paper, a cost optimization. Raw PDFs are stored in a dedicated S3 bucket.

Cross-cutting concerns. Amazon CloudWatch collects structured JSON logs and emits alarms on Lambda error rates and dead-letter-queue depth; alarms notify the team via Amazon Simple Notification Service. IAM roles are scoped to least privilege on a per-function basis.

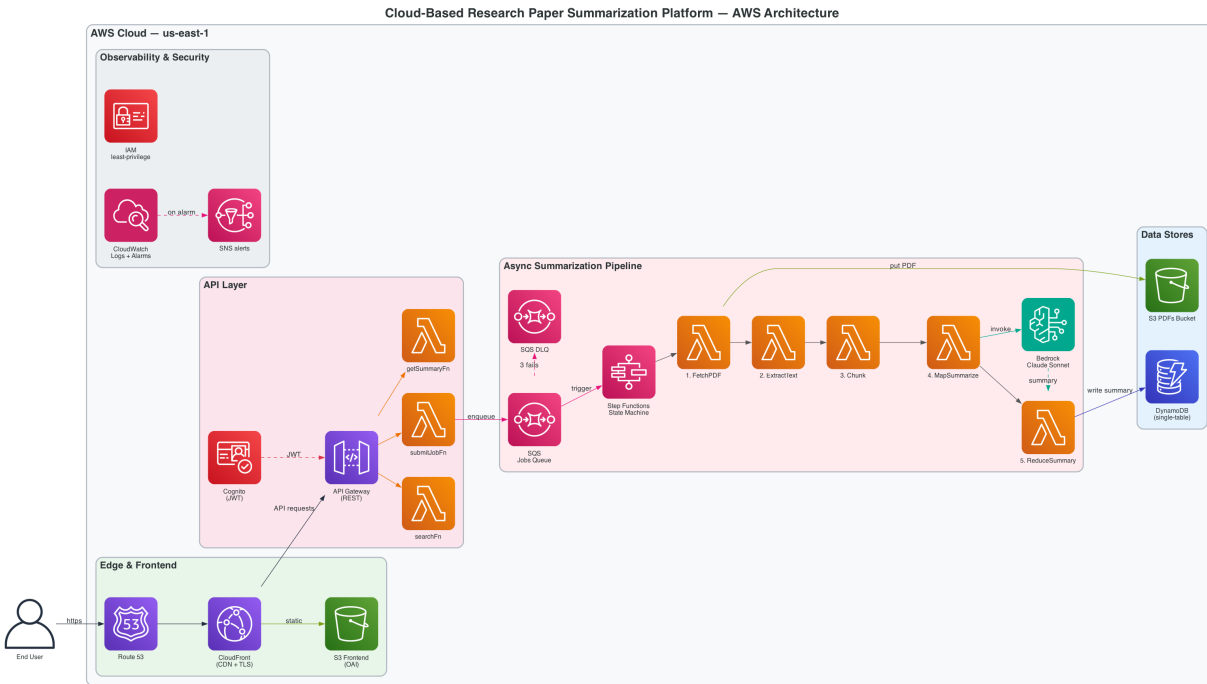


Figure 1: AWS deployment topology for the summarization platform.

6. Mapping to the AWS Well-Architected Framework

Table 1 maps the design to the six pillars of the AWS Well-Architected Framework.

Mapping of design elements to the Well-Architected pillars.

Pillar	Implementation
Operational Excellence	AWS CDK for Infrastructure-as-Code; CloudWatch logs and alarms; Step Functions visual execution history
Security	Cognito JWT Authorizer at API Gateway; least-privilege IAM roles; S3 Origin Access Identity; TLS in transit; encryption at rest
Reliability	Multi-Availability-Zone managed services; Step Functions step-level retries; SQS dead-letter queue; idempotent handlers
Performance Efficiency	Serverless autoscaling; parallel map-summarize; CloudFront edge caching
Cost Optimization	Pay-per-use serverless; content-hash summary deduplication; zero idle infrastructure
Sustainability	Minimized idle compute; deduplication avoids redundant LLM inference

7. DynamoDB Access Patterns

Table 2 documents the primary access patterns for the single-table design.

Primary DynamoDB access patterns.

Operation	Key Condition	Notes
Retrieve user profile	PK = USER#{u} AND SK = PROFILE#	Single-item fetch
List jobs for user	PK = USER#{u} AND begins_with(SK, JOB#)	Paged query
List summaries for user	PK = USER#{u} AND begins_with(SK, SUMMARY#)	Ordered by timestamp
Lookup existing summary by paper	GSI1PK = PAPER#{doi}	Deduplication

8. Service-Level Objectives

Preliminary service-level objectives.

Metric	Target
Search endpoint latency (P95)	Less than 1.5 seconds
Summary completion for a paper of up to 30 pages (P95)	Less than 90 seconds
API availability (rolling 30-day)	At least 99.5 percent
Summary generation success rate	At least 97 percent

9. Individual Contributions

Week 2 individual contributions.

Member	Focus
Shreyas Sankpal	Step Functions state-machine design
Pranjal Mishra	Architecture document and Well-Architected mapping
Yang Zheng	DynamoDB single-table schema and access patterns
Kerry Huang	Cognito and API Gateway authorizer design

10. Planned Work for Week 3

Week 3 will establish the project's infrastructure baseline and the first end-to-end deployment.

1. Initialize the AWS CDK project; commit baseline stacks for Amazon Cognito, S3, CloudFront, and API Gateway.
2. Deploy a GET /health endpoint backed by a stub Lambda, reachable through API Gateway with Cognito authorization enforced.
3. Establish the CloudWatch structured-log format and create the team CloudWatch dashboard.
4. Prototype the Step Functions state machine with a mocked Bedrock invocation on a single sample paper.